

L08: Introduction to Deep Learning for Computer Vision

CSci 560 Deep Learning w/ Python (Chollet) Ch. 8

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

L08.1 Introduction to Convnets

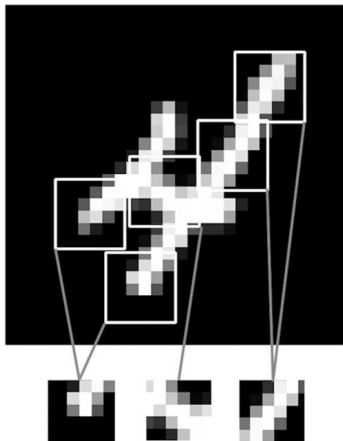
CSci 560: Deep Learning w/ Python (Chollet) Ch. 8.1

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

The Convolution Operation



The fundamental difference between a densely connected layer and a convolution layer is this:

- Dense layers learn global patterns in their (flattened vector) input feature space
- Conv2D layers learn local patterns, in the case of images, patterns found in small 2D windows of the input.

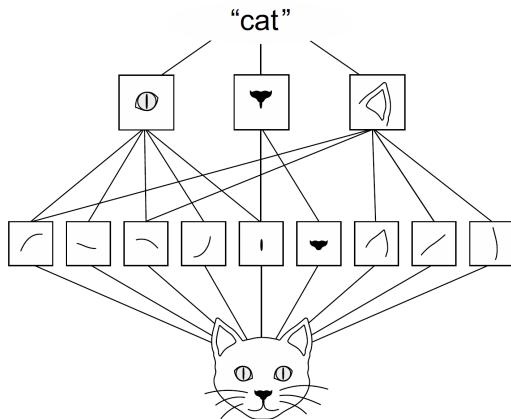
Figure 1: Images can be broken into local patterns such as edges, textures, and so on. (Chollet, 2021, pg.204)

First Key Property of convnets - Translation-Invariance

The patterns that a convnet learns are **translation-invariant**.

- After learning a pattern in the lower-right corner, a convnet can recognize it anywhere.
- This makes convnets data-efficient because *the visual world is fundamentally translation invariant*.

Second Key Property of convnets - Spatial Hierarchies



Convnets are capable of learning **spatial hierarchies** of patterns.

- A first convolutional layer will learn small patterns such as edges
- A second conv layer learns larger patterns made of the first layer patterns (like eyes or ears in the figure).

This allows convnets to efficiently learn increasingly complex and abstract visual concepts because *the visual world is fundamentally spatially hierarchical*.

Figure 2: The visual world forms a spatial hierarchy of visual modules: elementary lines or textures combine into simple objects such as eyes or ears, which combine into high-level concepts such as "cat". (Chollet, 2021, pg.204)

The Convolution Operation (again)

- Convolutions operate over rank-3 tensors called **feature maps**
 - two spatial axes (height and width)
 - depth axis
- Convolution operation extracts patches (`kernel_size`) and applies same transformation to all these patches.
- The output feature map is a rank-3 tensor
 - it has width and height
 - Its depth is arbitrary (`filters`)
- **Filters** encode specific aspects of the input data, and detect it in a patch or not.
- At a high level a single filter could encode the concept “presence of a face”

MNIST Convolutions

- First convolution layer takes input feature map of size (28, 28, 1)
- Outputs feature map of size (26, 26, 32)
- It computes 32 filters over its input.
- Each of these 32 output channels contains a 26x26 grid of values
 - which is a **response map** of the filter over the input, indicating response of that pattern at image locations.

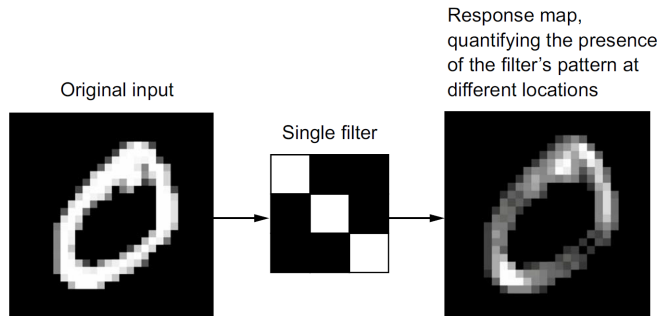


Figure 3: The concept of a response map: a 2D map of the presence of a pattern at different locations in an input. (Chollet, 2021, pg.204)

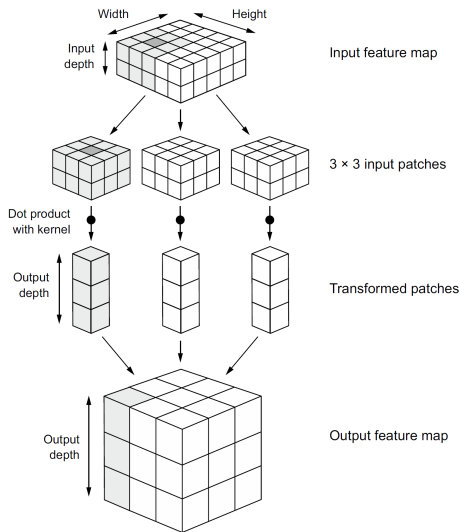
Feature Maps

- **Feature map:** every dimension in the depth axis is a *feature* or *filter*
- The rank-2 tensor output $[:, :, n]$ is the 2D spatial *map* of the response of this filter over the input.

Convolutions are defined by two key parameters:

- 1 Size of the patches extracted from inputs - typically 3x3 or 5x5.
- 2 Depth of the output feature map - number of filters/features discovered by the convolution.

How Convolution Works



- Slide window (e.g. 3x3) over the 3D input feature map.
 - Each such 3D patch is transformed into a 1D vector
 - by tensor product of **convolution kernel** with the patch
 - All vectors (one per patch) are spatially reassembled into 3D output map
- Note: output widthxheight is 3x3 because of the sliding

Understanding Border Effects

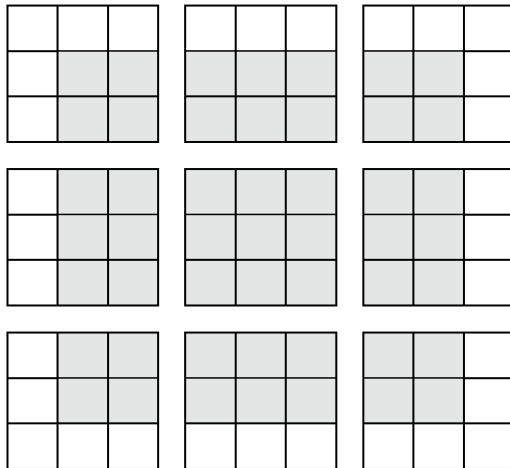
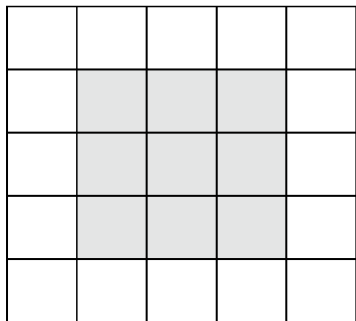


Figure 5: Valid locations of 3x3 patches in a 5x5 input feature map. (Chollet, 2021, pg.208)

Padding and Striding

- **Padding** consists of adding an appropriate number of rows and columns on each side of the input feature map so as to fit center convolution windows around every input tile.
- **Strided convolutions**: distance between two successive windows
 - Using stride 2 means the width and height of the feature map are downsampled by a factor of 2
- In classification models, instead of strides, tend to use the **max-pooling** operation to downsample feature maps.

Add Padding to Keep Spatial Dimensions

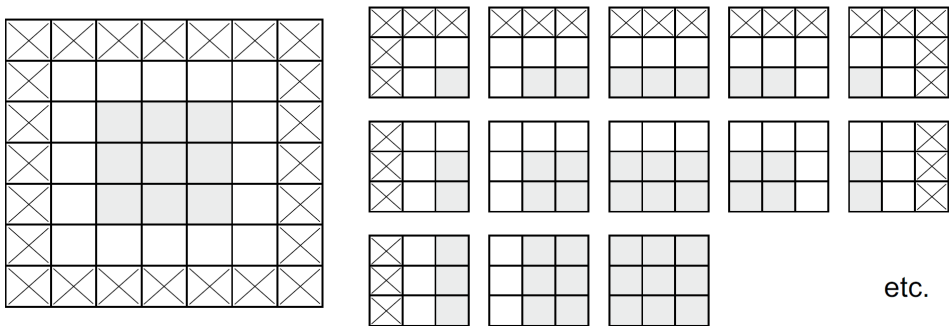


Figure 6: Padding a 5x5 input in order to be able to extract 25 3x3 patches. (Chollet, 2021, pg.208)

Understanding Convolution Strides

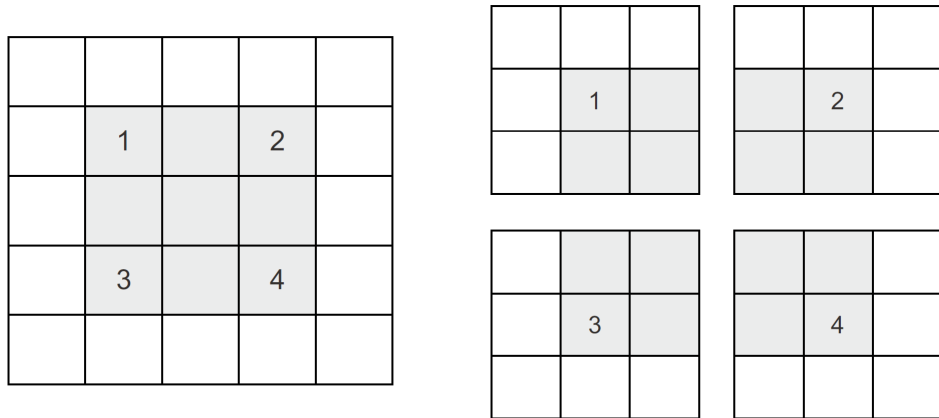


Figure 7: 3x3 convolution patches with 2x2 strides. (Chollet, 2021, pg.209)

The Max-pooling Operation

- You may have noticed that the size of the feature maps is halved after every `MaxPooling2D` layer.
- The role of max pooling: to aggressively downsample feature maps.
- Similar to convolutions, slide windows and transform local patches. But a simple max is done rather than a tensor transformation
- Usually use max pooling with 2x2 window and stride 2.
- Usually use convolution with 3x3 window and no stride (stride 1).

Summary of Convnets, Convolutions and Max Pooling

- The visual world is fundamentally **translation-invariant**, convolutions learn translation-invariant kernels.
- The visual world is fundamentally **spatially hierarchical**, successive convolution layers are capable of learning increasingly high-level spatial hierarchies.
- Convolution layers operate by sliding a window to extract patches, and applies convolution kernel learned to each pass.
- Every dimension in the depth axis from a convolution is a **feature map**.
- The role of max pooling is to aggressively downsample feature maps.

L08.2 Training a Convnet from Scratch on a Small Dataset

CSci 560: Deep Learning w/ Python (Chollet) Ch. 8.2

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

L08.3 Leveraging a Pretrained Model

CSci 560: Deep Learning w/ Python (Chollet) Ch. 8.3

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

Leveraging Pretrained Models Summary

- Pretrained models on large imaging datasets are available through the Keras API and from other methods.
- The convolutional layers of such pretrained models have learned a powerful and useful hierarchy of features, from the low level to the more abstract.
- Leveraging a pretrained model can help get excellent image classification results if even only have a (relatively) small set of images to train with.
- There are two basic approaches to using a pretrained model
 - ① Extract features by passing inputs into pretrained model and gathering last convolutional layer outputs. This is computationally faster, but you can't use data augmentation or other fine tuning techniques with this approach.
 - ② Freeze the weights of the pretrained model and add in your data augmentation stage before and classifier after, and retrain from begin to end using your data augmentation. Computationally more expensive but can be fine-tuned better.
- Can also help to sometime first train a model with frozen pretrained weights, then unfreeze some of the top layers of the pretrained model and train some more.

Summary of Introduction to Deep Learning for Computer Vision

- Convnets are the best type of machine learning models for computer vision tasks. It's possible to train one from scratch even on a very small dataset, with decent results.
- Convnets work by learning a hierarchy of modular patterns and concepts to represent the visual world.
- On a small dataset, overfitting will be the main issue. Data augmentation is a powerful way to fight overfitting when you're working with image data.
- It's easy to reuse an existing convnet on a new dataset via feature extraction. This is a valuable technique for working with small image datasets.
- As a complement to feature extraction, you can use fine-tuning, which adapts to a new problem some of the representations previously learned by an existing model. This pushes performance a bit further.

Feature Extraction with a Pretrained Model

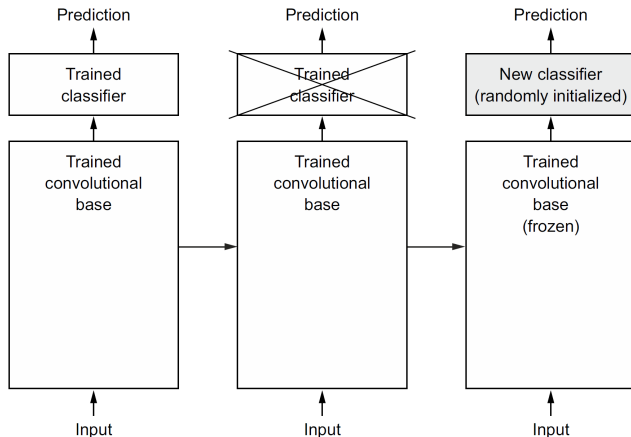


Figure 8: Swapping classifiers while keeping the same convolutional base. (Chollet, 2021, pg.204)

- Feature extraction consists of using the representations learned by a previously trained model to extract interesting features from new samples.
- Convnets for image classification have 2 parts
 - 1 series of pooling and convolution layers (feature hierarchy)
 - 2 densely connected final classifier
- The feature hierarchy is called the **convolutional base** of the model
- It has prelearned low to high level features that can be reused.



Chollet, F. (2021). *Deep learning with python* (second). [Manning](#).